데이터분석을위한 PANDAS

PANDAS PROGRAMMING

PANDAS 인덱스 활용

인덱스 활용

◆ 데이터프레임(DataFrame)

- 특정 열을 행 인덱스로 설정

```
DataFrame객체.set_index( [ '열이름'] )

DataFrame객체.set_index( '열이름' )
```

→ 원본 데이터 유지, 변경된 객체 반환

→ inplace=True : 원본 데이터 변환

인덱스 활용

◆ 데이터프레임(DataFrame)

행 인덱스 설정

```
import pandas as pd

# DataFrame() 함수로 데이터프레임 변환. 변수 df에 저장
exam_data = { '이름' : [ '서준', '우현', '인아'],
              '수학' : [ 90, 80, 70],
              '영어' : [ 98, 89, 95],
              '음악' : [ 85, 95, 100],
              '체육' : [ 100, 90, 90]}
df = pd.DataFrame(exam_data)

# 특정 열(column)을 행 인덱스(index)로 설정
ndf1 = df.set_index( ['이름'] )

ndf2 = ndf.set_index( '음악' )

ndf3 = ndf.set_index( ['수학', '음악'] )
```

인덱스 활용

◆ 데이터프레임(DataFrame)

▪ 행 인덱스 재배열

DataFrame객체.reindex(새로운 인덱스 배열, fill_value=값)

→ 원본 데이터 유지, 변경된 객체 반환

→ inplace=True : 원본 데이터 변환

→ fill_value= 데이터가 없는 경우 기본 값 NaN

인덱스 활용

◆ 데이터프레임(DataFrame)

행 인덱스 재배열

```
# 딕셔서리 정의
dict_data = { 'c0':[ 1,  2,  3], 'c1':[  4,  5,  6], 'c2':[ 7, 8, 9],
              'c3':[10,11,12],  'c4':[13,14,15]}

# 인덱스를 [r0, r1, r2]로 지정
df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])

# 인덱스 재지정
new_index = ['r0', 'r1', 'r2', 'r3', 'r4']
ndf = df.reindex( new_index )

# reindex로 발생한 NaN값을 숫자 0으로 채우기
new_index = ['r0', 'r1', 'r2', 'r3', 'r4']
ndf2 = df.reindex( new_index, fill_value=0 )
```

인덱스 활용

◆ 데이터프레임(DataFrame)

- 행 인덱스 초기화 ➔ 정수형 인덱스로 초기화

```
DataFrame객체.reset_index(  )
```

→ 정수형 위치 인덱스로 초기화

→ 기존 행 인덱스 => 열(column)으로 이동

인덱스 활용

◆ 데이터프레임(DataFrame)

행 인덱스 초기화

```
# 딕셔서리 정의
dict_data = {'c0':[1,2,3],      'c1':[4,5,6], 'c2':[7,8,9],
             'c3':[10,11,12], 'c4':[13,14,15]}

# 데이터프레임으로 변환. 인덱스 [r0, r1, r2]로 지정
df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])

# 행 인덱스를 정수형으로 초기화
ndf = df.reset_index( )
```

인덱스 활용

◆ 데이터프레임(DataFrame)

▪ 열 이름 기준 정렬

```
DataFrame객체.sort_index( axis=0, ascending=True )
```

→ 행을 기준으로 정렬

→ inplace = False : 원본 적용 여부 설정

→ na_position = 'last' : 빈칸(NaN) 위치

인덱스 활용

◆ 데이터프레임(DataFrame)

행 인덱스 정렬

```
# 데이터 정의
dict_data = {'c0':[1,2,3], 'c1':[4,5,6], 'c2':[7,8,9],
             'c3':[10,11,12], 'c4':[13,14,15]}

# 데이터프레임으로 변환. 인덱스 [r0, r1, r2]로 지정
df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])

# 내림차순으로 행 인덱스 정렬
ndf = df.sort_index(ascending=False)
```

인덱스 활용

◆ 데이터프레임(DataFrame)

▪ 열 이름 기준 정렬

```
DataFrame객체.sort_values( by='열이름', ascending=True )
```

→ 열을 기준으로 정렬

→ inplace = False : 원본 적용 여부 설정

→ na_position = 'last' : 빈칸(NaN) 위치

인덱스 활용

◆ 데이터프레임(DataFrame)

열 이름 정렬

```
import pandas as pd

# 딕셔서리를 정의
dict_data = { 'c0':[1,2,3], 'c1':[4,5,6], 'c2':[7,8,9],
              'c3':[10,11,12], 'c4':[13,14,15] }

# 데이터프레임으로 변환. 인덱스 [r0, r1, r2]로 지정
df = pd.DataFrame(dict_data, index=['r0', 'r1', 'r2'])

# c1 열 기준으로 내림차순 정렬
ndf = df.sort_values(by='c1', ascending=False)
print(ndf)
```

PANDAS 연산

산술 연산

◆ 연산 프로세스

행/열 인덱스 기준 정렬 → 동일 위치 원소 일대일 대응 → 일대일 대응 원소끼리 연산

산술 연산

◆ Series 연산

숫 자 연 산	Series객체 + 연산자(+, - , * , /) + 숫자
시리즈 연 산	Series객체 + 연산자(+, - , * , /) + Series객체
연 산 메소드	Series객체.연산메소드(Series객체, fill_value=0)

산술 연산

◆ Series 연산

Series 연산

```
import pandas as pd

# 시리즈 만들기
student1 = pd.Series({'국어':100, '영어':80, '수학':90})
print(student1)

# 학생의 과목별 점수를 200으로 나누기
#  +, - , *  10으로 처리 하기
percentage = student1 / 200      # Series Obj / 숫자

print(percentage)
print(type(percentage))
```

```
국어    100
영어     80
수학     90
dtype: int64


국어    0.50
영어    0.40
수학    0.45
dtype: float64
```

산술 연산

◆ Series 연산

Series 연산

```
import pandas as pd

# 딕셔너리 데이터로 판다스 시리즈 만들기
student1 = pd.Series({'국어':100, '영어':80, '수학':90})
student2 = pd.Series({'수학':80, '국어':90, '영어':80})

# 두 학생의 과목별 점수로 사칙연산 수행
addition       = student1 + student2          #덧셈
subtraction    = student1 - student2          #뺄셈
multiplication = student1 * student2          #곱셈
division       = student1 / student2          #나눗셈
print(type(division))
```

산술 연산

◆ Series 연산

Series 연산

```
# 사칙연산 결과를 데이터프레임으로 합치기
#  (시리즈 -> 데이터프레임)
result = pd.DataFrame( [ addition, subtraction,
                         multiplication, division],
                         index=['덧셈', '뺄셈', '곱셈', '나눗셈'])
print(result)
```

산술 연산

◆ Series 연산

Series 연산

```
# 두 학생의 과목별 점수로 사칙연산 수행 (연산 메소드 사용)
# 공백 인덱스 , NaN 연산 피하기 ➔ fill_value
sr_add = student1.add(student2, fill_value=0)    #덧셈
sr_sub = student1.sub(student2, fill_value=0)    #뺄셈
sr_mul = student1.mul(student2, fill_value=0)    #곱셈
sr_div  = student1.div(student2, fill_value=0)    #나눗셈

# 사칙연산 결과 합치기 (시리즈 -> 데이터프레임)
result = pd.DataFrame([sr_add, sr_sub, sr_mul, sr_div],
                      index=['덧셈', '뺄셈', '곱셈', '나눗셈'])
```

산술 연산

◆ DataFrame 연산

숫 자 연 산	DataFrame객체 + 연산자(+, - , * , /) + 숫자
시리즈 연 산	DataFrame객체 + 연산자(+, - , * , /) + DataFrame객체
연 산 메소드	DataFrame객체.연산메소드(DataFrame객체, fill_value=0)

산술 연산

◆ DataFrame 연산

DataFrame 연산

```
# 모듈 로딩 ----------------------------------------
import pandas as pd

exam_data = { '이름' : [ '서준', '우현', '인아'],
              '수학' : [ 90, 80, 70],
              '영어' : [ 98, 89, 95],
              '음악' : [ 85, 95, 100],
              '체육' : [ 100, 90, 90]}

df = pd.DataFrame(exam_data)
df.set_index('이름', inplace=True)

# 데이터프레임에 숫자 2 곱하기
df2 = df * 2
print(df2)
print(type(df2))
```

산술 연산

◆ DataFrame 연산

DataFrame 연산

```
# 데이터프레임 - 데이터프레임
sub = df2 - df
print(sub)
print(type(sub))

# 데이터프레임 + 데이터프레임
add = df2 + df
print(add)
print(type(add))
```